

프로젝트 준비

프로젝트에 필요한 내용들

준비물 ① — 보안그룹 규칙

- **ingress** — 어제 콘솔의 "인바운드 규칙" 탭. 밖에서 안으로 들어오는 트래픽 허용
- **egress** — 어제 콘솔의 "아웃바운드 규칙" 탭. 안에서 밖으로 나가는 트래픽 허용
- **왜 새 문법이 아닌가** — aws_security_group 자원이 요구하는 정해진 하위 블록 이름일 뿐. 안의 구조는 문법②(이름=값) 그대로
- **egress는 보통 통째로 복붙** — 나가는 트래픽까지 세세히 막는 경우는 드물어 "전체 허용" 형태가 관례

ingress / egress 한 줄씩 읽기

```
ingress {  
  from_port    = 80           # 80번 포트로  
  to_port      = 80           # (80번부터 80번까지 = 80번만)  
  protocol     = "tcp"        # TCP로  
  cidr_blocks  = ["0.0.0.0/0"] # 전 세계 어디서든  
}  
  
egress {  
  from_port    = 0           # 전체 포트  
  to_port      = 0           # 전체 포트  
  protocol     = "-1"        # 전체 프로토콜  
  cidr_blocks  = ["0.0.0.0/0"] # 어디로든 나갈 수 있다  
}
```

왼쪽 이름(from_port, protocol...)은 정해진 것 — 자유롭게 짓는 건 오직 블록의 "별명"뿐 (문법① 복습)

준비물 ② — user_data: 서버가 "스스로" 실행하는 스크립트

- **정체** — EC2가 처음 부팅될 때, 딱 한 번, 자동으로 실행되는 셸 스크립트
- **어디서 봤나** — 어제 콘솔의 "고급 세부 정보 > 사용자 데이터" 입력칸과 완전히 같은 기능. 실습 4에서 이미 한 번 사용
- **세 가지 특성** — ① 최초 부팅 시 1회만 실행 (재부팅해도 다시 안 함) ② root 권한이라 sudo 불필요 ③ 완료까지 1~2분 걸림
- **프로젝트에서** — 이 스크립트 안에 Docker 설치 + nginx 실행까지 전부 넣는다 — apply 한 번으로 배포가 끝나는 이유

user_data 문법 — heredoc (<<-EOF ~ EOF)

```
user_data = <<-EOF
#!/bin/bash
dnf install -y docker
systemctl start docker
docker run -d -p 80:80 nginx
EOF

# <<-EOF 로 시작해서 EOF 로 끝날 때까지가 전부 하나의 긴 문자열
# "이 안에는 줄바꿈 포함 텍스트를 통째로 넣겠다"는 뜻
# EOF는 이름일 뿐 – END든 SCRIPT든 시작·끝만 같으면 아무 단어나 OK
#
# 안의 내용은 새 문법이 아니라 그냥 bash 명령 나열
# (dnf install, systemctl, docker run – 전부 Day2에서 이미 친 명령)
```

오늘 문법③가지에 이 heredoc까지 더하면, 프로젝트에 필요한 문법은 전부 끝난다

준비물 ③ — docker run 복습 (Day2 오후에 이미 배움)

- **docker run -d -p 80:80 nginx** — 이 한 줄이 전부. -d는 백그라운드 실행, -p 80:80은 포트 연결
- **nginx가 뭔가** — Docker Hub의 공개 이미지 — 별도 로그인 없이 누구나 pull 가능 (ECR과 달리 인증 불필요)
- **그래서 이번 프로젝트엔** — IAM 역할도, ECR 로그인도 필요 없다 — Terraform 문법 연습에만 집중 가능
- **Day2와 다른 점 딱 하나** — 어제는 사람이 터미널에 직접 쳤고, 오늘은 user_data 안에서 서버가 스스로 친다

프로젝트— "Terraform으로 Docker Nginx 웹 서비스"

- **미션** — apply 한 번으로: VPC부터 새로 만들고 → 그 안의 서버가 스스로 Docker를 설치하고 → nginx를 실행해서 → 브라우저에 nginx 기본 페이지가 보이게 하라
- **조합하는 것** — 오늘 실습 4(VPC·서브넷·IGW·라우팅·보안그룹) + Day2 오후(docker run 명령) + 방금 배운 heredoc
- **출발점** — 실습 4의 vpc.tf 구조를 그대로 복사해서 시작 (새 파일 proj.tf 권장 — 별명이 겹치지 않게 vpc_web 대신 새 이름 쓰기)
- **바꿀 곳은 딱 한 곳** — EC2의 user_data 안 내용만 httpd 설치에서 docker+nginx 실행으로 교체
- **성공 기준** — output으로 나온 주소를 브라우저로 열었을 때 "Welcome to nginx!" 페이지가 보이면 됩니다.

프로젝트 — 힌트 1/2 (네트워크는 실습 4와 동일)

```
# VPC · 서브넷 · IGW · 라우팅 · 보안그룹은
# 실습 4의 vpc.tf 블록을 거의 그대로 복사해서 쓰면 된다
# (별명만 겹치지 않게 바꾸기 - 예: my_vpc → proj_vpc)

resource "aws_vpc" "proj_vpc" { ... }
resource "aws_subnet" "proj_public" { ... }
resource "aws_internet_gateway" "proj_igw" { ... }
resource "aws_route_table" "proj_rt" { ... }
resource "aws_route_table_association" "proj_assoc" { ... }

resource "aws_security_group" "proj_web" {
  # ingress: 80 포트, 0.0.0.0/0 (준비물 ①에서 배운 그대로)
  # egress: 전체 허용
}
```

복붙 후 vpc_id 등 참조하는 부분(문법③)의 별명도 함께 바뀌었는지 확인!

프로젝트 — 힌트 2/2 (바뀌는 곳은 user_data뿐)

```
resource "aws_instance" "proj_web" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.proj_public.id
  vpc_security_group_ids = [aws_security_group.proj_web.id]

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y docker
    systemctl start docker
    systemctl enable docker
    docker run -d -p 80:80 nginx
  EOF

  tags = { Name = "proj-nginx-${var.student_id}" }
}

output "nginx_url" {
  value = "http://${aws_instance.proj_web.public_ip}"
}
```

실습 4의 httpd 3줄이 docker 4줄로 바뀐 것 — 나머지 블록은 손댈 필요 없음

프로젝트 — 자주 막히는 지점

- **apply는 성공했는데 페이지가 안 뜬다** — 정상! docker 설치+이미지 다운로드에 1~2분 걸림. 잠시 후 새로고침
- **그래도 안 뜬다면** — 보안그룹에 80 ingress가 진짜 있는지, subnet_id·security_group 참조가 proj_* 별명으로 맞게 연결됐는지 확인
- **"Reference to undeclared resource"** — 복사하며 별명을 바꾸다가 한 군데를 놓친 것 — 문법③ 참조들을 전부 새 별명으로 통일했는지 확인
- **VPC 개수 오류** — 이 프로젝트도 VPC를 새로 만든다 — 실습 4의 VPC를 아직 안 지웠다면 먼저 정리 고려

프로젝트 제출

- **terraform으로 생성한 내용을 스크린샷** — terraform code 메모장에 저장, nginx 홈페이지, ec2 를 스크린샷으로 저장
- **제출하는 곳**— shjeon0708@gmail.com